

■目次／シート名一覧

[目的](#)

[ffi-napiサーバー8033](#)

[ffi-napiサーバー80](#)

[JSONプロトコル](#)

[付録1. エラーコード](#)

[付録2. RUN_BAT](#)

■目的

- 最新のNodeJS-LTSでは、32ビット版は提供されなくなった。
32ビット版のNodeJSの最終バージョンはv22 で、2026年4月でサポート期限切れ。
- 近い将来、フロントエンドはAIが生成ようになる可能性があり、
生成AIの実績豊富なのは、最新のLTSであると思われる。
- このため、フロントエンド開発は最新の64bit NodeJS LTS環境にしておきたい。
- 一方で、業務の現場では、32bit-DLLは安定しており捨て難い。

⇒フロントエンドとバックエンドを分けて、クライアント/サーバーモデルで連携する。

(1)64bit Electron 最新LTSをフロントエンドとし、
バックエンドをffi-napi サーバー(port8033版)とする構成。

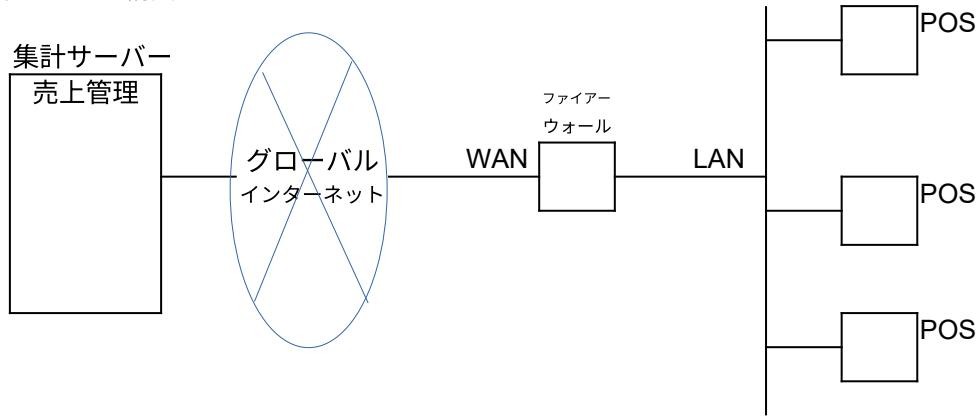
(2)Chromeなどのブラウザーをフロントエンドとし、
バックエンドをffi-napiサーバー(port80版) とする構成。
(注)生成AIの実績は、Electronよりも一般ブラウザが先行すると予想されるため、
このタイプ2も想定に入れた。

現時点で、これら 2 タイプのffi-napiバックエンドサーバーの開発を
本プロジェクトの目的とする。

また本プロジェクトの目的外ではあるが、将来、タイプ(1)、タイプ(2)のffi-napiサーバーを
ビジネス用途に通信プロトコルに互換性を持ったC言語で堅牢に実装することもかんがみ、
本プロジェクトは機能仕様を洗い出すためのプロトタイプという位置づけとする。

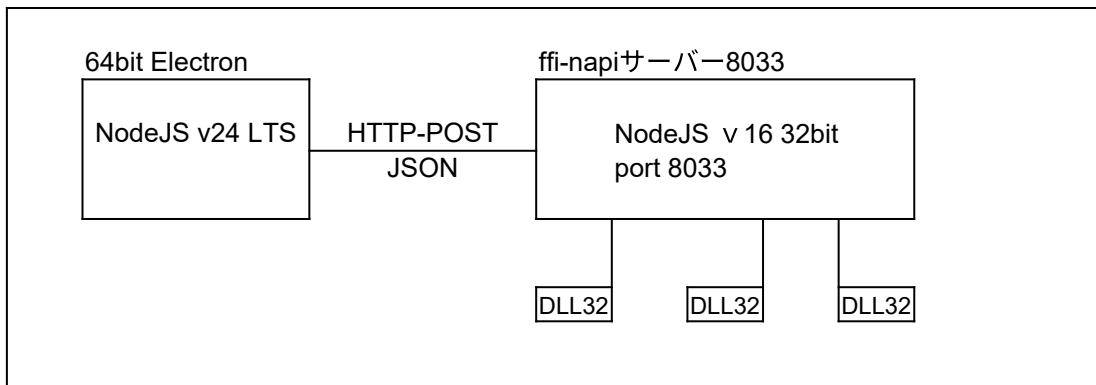
目的・シート

■全体システム構成



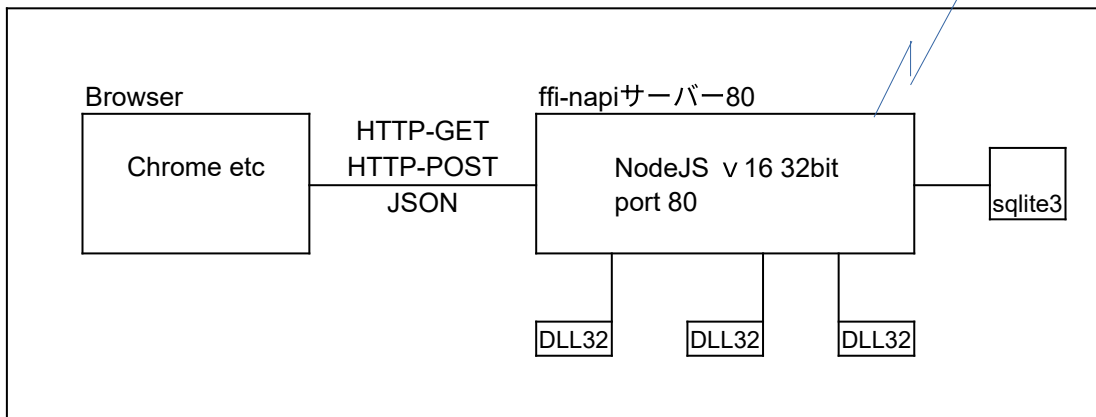
■POS構成タイプ1（フロントエンド==Electron）

localhost



■POS構成タイプ2（フロントエンド==Browser(Chrome等)）

localhost



目的・シート

■GitHub提供ソースコードのディレクトリ構成

APP_TOP

```
ffi_honji
d  |bin
   |f    busybox.exe

d  node-v16.20.2-win-x86
d  node-v24.18.0-win-x64
f  node16.bat
f  node24.bat
d  client/Electron1
d  device_dll
d  doc
d  server
d  server_port80
```

git clone したもの
ユーティリティを格納する
<https://frippersy.org/busybox/> からダウンロード

node-v16.20.2-win-x86.zip を解凍
node-v24.18.0-win-x64.zip を解凍
node16用コマンドプロンプト
node24用コマンドプロンプト
サンプルElectronコード8033
テスト用DLL開発環境
ドキュメント
ffi-napiサーバー8033（タイプ1）
ffi-napiサーバー80（タイプ2）

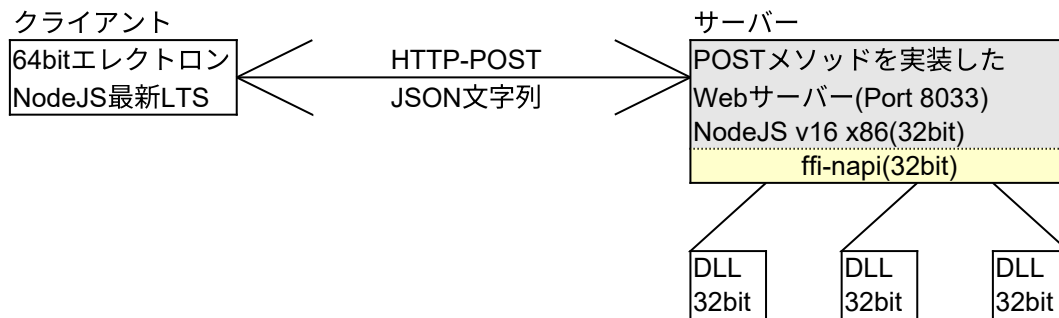
ffi-napiサーバー(port8033版)

■目的

64bitエレクトロン向けにレガシー32bitDLL呼び出し機能をサーバーとして提供する。

NodeJSでレガシー32bitDLLを呼び出す機構は ffi-napiが最もネットの情報量が多い。
この ffi-napi が安定動作するのは NodeJS x86 v16 となる。

■システム構成



■フォルダ構成

APP_TOP

```

d  server
  d  dll
  d  logs
  d  node_modules
  f  ffi_honji_svr.js
  f  index.js
  f  run.bat
    
```

フォルダトップ

装置DLL置き場
ログ置き場
NodeJSモジュール(v16で構築)
ffi_honji本体(ffi-napiサーバー本体)
エントリープログラム
ffi-napiサーバー起動バッチ

■起動コマンド

(1)オーソドックスな起動方法

```
$ cd server  
$ node index.js
```

(2)死活監視バッチからの起動

```
$ cd server  
$ run.bat
```

■停止方法

- (1)ffi-napiサーバーが起動に成功するとlogsフォルダ下にkill_svr.bat を生成する。
- (2)この kill_svr.bat を実行すると、サーバーは終了する。
- (3)死活監視バッチからの起動中に kill_svr.bat を実行すると、死活監視も終了する。
- (4)死活監視バッチは活かして、サーバーだけ終了をかけたいときは、
logs/pid.txt にサーバーのプロセスIDが記録してあるので、それを見て、
コマンドプロンプト (別窓)から
\$ kill /f /pid 1234
のようにサーバーにシグナルを送る。
約5秒後に、死活監視バッチからサーバー再起動がかかる。

■運用環境への展開

- (1)serverフォルダー一式に、node.exe を含めて、任意の運用フォルダーに展開。
- (2)run.bat のショートカットリンクをデスクトップ上に作り、
run.batショートカットをエクスプローラのアドレスバー、shell:startup に移動する。
- (3)これで、Windowsログイン時にffi-napiサーバーが起動するようになる

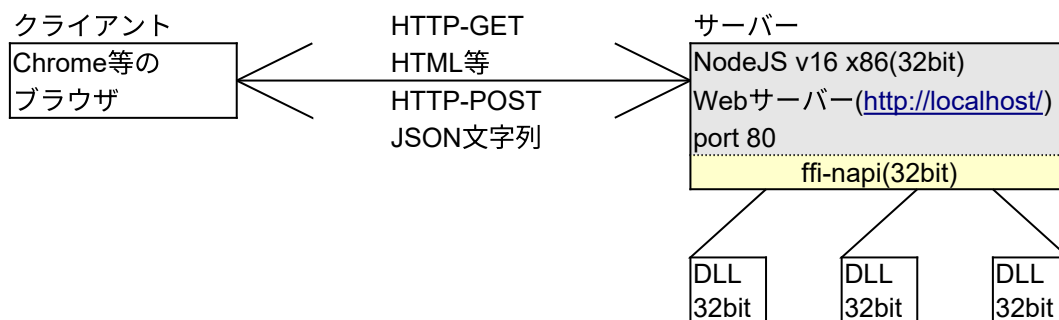
ffi-napiサーバー(port80版)

■目的

Chromeなどの一般ブラウザ向けにWebサーバー機能および、
レガシー32bitDLL呼び出し機能をサーバーとして提供する。

NodeJSでレガシー32bitDLLを呼び出す機構は ffi-napiが最もネットの情報量が多い。
この ffi-napi が安定動作するのは NodeJS x86 v16 となる。

■システム構成



■フォルダ構成

APP_TOP

d	server_port80	ffi-napiサーバー80
d	dll	装置DLL置き場
d	data	dataフォルダ(CSV,sqlite3ファイル等)
d	share	集計サーバーとの共有フォルダ
f	htdocs.zip	(3) 集計サーバーからダウンロードしたhtdocs.zip
d	logs	ログ置き場
f	pid.txt	(4) サーバーのプロセスIDを保持。
f	svr_kill.bat	(5) サーバーを強制終了するバッチコマンド
f	log_yyyymmdd_hhnnss.txt	(6) ログファイル
f	start_log.txt	(7) サーバー開始およびソフトリセットの記録。
d	htdocs	(11) DOCUMENT_ROOT
f	htdocs.zip	(12) zip形式のWebドキュメント複合体
f	ffi_honji_svr.js	ffi_honji本体(ffi_napiサーバー本体)
f	index.js	エントリプログラム
f	run80.bat	(13) ffi-napiサーバー起動&死活監視バッチ
f	run80_nowin.vbs	run80.bat を窓無しで起動するVBスクリプト

ffi-napiサーバー80・シート

- (3) htdocs.zip 各号機間で共通配布するアプリの圧縮ファイル
試験環境のhtdocs をzip形式で圧縮したもの
Webサーバー起動時に、サーバープログラムが(3)→(12)へコピーする。
- (4) pid.txt ffi-napiサーバーのプロセスIDを保持する。
- (5) kill_svr.bat ffi-napiを外部から強制終了させるためのバッチコマンド
死活監視プログラムも終了する。
- (6) log_yyyymmdd_hhnnss.txt ffi-napiサーバーのコンソール出力を蓄積する。
- (7) start_log.txt ffi-napiサーバーの開始時刻(ソフト、ハードとも)を記録する。
- (11) htdocs Webサーバーの DOCUMENT_ROOT
本号機独自の処理があれば、このフォルダ内に記述する。
htdocs.zip よりも優先順位が高く処理を行う。
- (12) htdocs.zip 各号機間共通のDOCUMENT_ROOT圧縮ファイル
- (13) run80.bat ffi-napiの起動コマンド。RUN_BATシートにて詳述する。

■起動コマンド

(1)オーソドックスな起動方法

```
$ cd server  
$ node index.js
```

(2)死活監視バッチからの起動

```
$ cd server  
$ run80.bat
```

(3)死活監視バッチ（窓無し）からの起動

```
$ cd server  
$ run80_nowin.vbs
```

■停止方法

- (1)ffi-napiサーバーが起動に成功するとlogsフォルダ下にkill_svr.bat を生成する。
- (2)この kill_svr.bat を実行すると、サーバーは終了する。
- (3)死活監視バッチからの起動中に kill_svr.bat を実行すると、死活監視も終了する。
- (4)死活監視バッチは活かして、サーバーだけ終了をかけたいときは、
logs/pid.txt にサーバーのプロセスIDが記録してあるので、それを見て、
コマンドプロンプト（別窓）から
\$ kill /f /pid 1234
のようにサーバーにシグナルを送る。
約5秒後に、死活監視バッチからサーバー再起動がかかる。

■運用環境への展開

- (1)serverフォルダー一式に、node.exe を含めて、任意の運用フォルダーに展開。
- (2)run80_nowin.vbsのショートカットリンクをデスクトップ上に作成し、
そのショートカットをエクスプローラのアドレスバー、shell:startup に移動する。
- (3)これで、Windowsログイン時にffi-napiサーバーが起動するようになる（窓無し）

JSONプロトコル・シート

ffi_honjiクライアント/サーバー間のJSONプロトコル

機能番号	内容
1	ffi.Libraryの実行
2	DLL関数の呼び出し
3	本サーバー起動直後のffiロック解除(*)

(*)ffi_honjiサーバー起動直後は、機能番号1・2にロックがかかっておりNG応答となる。機能番号3でロックを解除して1・2が使用可能になる。

クライアント側はffiを利用しようとして、ロックNG状態になったら、サーバーがコントロールマスタによって再起動がかかったものと判断して、機能番号3でロックを解除し、装置DLL初期化シーケンスに入ることが要請される。

JSONプロトコル・シート

機能 1 (ffi.Libraryの実行要求)

要求

```
{  
  xcmdno      :      1  
  xdllnm      :      dllファイル名  
  xfuncs      :      ffi.Libraryの第 2 引数に渡す関数定義オブジェクト  
}
```

応答

```
{  
I   xcmdno      :      1  
I   xdllnm      :      dllファイル名  
I   xfuncs      :      ffi.Libraryの第 2 引数に渡す関数定義オブジェクト  
O   xlibno      :      ライブラリ番号(コマンド 2 に引き継ぐ)  
O   xstats      :      “OK” or “NG”  
O   xerrno      :      “OK”:0, “NG”:非 0  
O   xmesag      :      “OK”:”, “NG”:エラーメッセージ  
}
```

JSONプロトコル・シート

機能 2 (DLL関数の実行)

要求

```
{  
    xcmdno    :      1  
    xlibno    :      ライブラリ番号  
    xinarg    :      DLL関数に渡す配列  
}
```

応答

```
{  
I    xcmdno    :      1  
I    xlibno    :      ライブラリ番号  
I    xinarg    :      DLL関数に渡す配列  
O    xreslt    :      DLL関数の戻り値、内部エラー時 9999  
O    xotarg    :      DLL関数のOUTPUT引数  
O    xstats    :      “OK” or “NG”  
O    xerrno    :      “OK”:0, “NG”:非 0  
O    xmesag    :      “OK”:””, “NG”:エラーメッセージ  
}
```

JSONプロトコル・シート

機能3（サーバー起動直後のfpiロック解除)

要求

```
{  
    xcmdno    :      3  
}
```

応答

```
{  
O   xstats    :      “OK” or “NG”  
O   xerrno    :      “OK”:0, “NG”:非0  
O   xmesag    :      “OK”:””, “NG”:エラーメッセージ  
}
```

エラーコード表

- | | |
|---|------------------------|
| 1 | JSON parse error |
| 2 | メンバー変数が見つからない。 |
| 3 | 関数定義の事前審査エラー |
| 4 | ffi.Library呼出エラー |
| 5 | xinarg(xcmdno=2)パースエラー |
| 6 | xotarg(xcmdno=2)パースエラー |
| 7 | api is locked. |
| 8 | 予約エラー |

付録 server_port80 起動バッチ

概要 server_port80を起動するとともに、死活監視の役割も持つ。
server_port80が異常終了すると、5秒後に再起動をかける。

ソースコード

```
1  @echo off
2  echo "this application is for node_v16_x86"
3  set CWD=%~dp0
4  if "%APP_TOP%"==" " set APP_TOP=%CWD%..\
5
6  :START
7  cd %CWD%
8  where node
9  if %errorlevel% equ 0 (
10   for /f "tokens=" %%i in ('node --version') do set NODE_VER=%%i
11   for /f "tokens=1,2 delims=v." %%a in ('node --version') do (
12     set NODE_MAJOR_VERSION=%%b
13   )
14   if "%NODE_MAJOR_VERSION%"=="16" (
15     set TARGET=node.exe
16     node.exe index.js
17   ) else (
18     set TARGET=%APP_TOP%node16.bat
19     call %APP_TOP%node16.bat index.js
20   )
21 ) else (
22   set TARGET=%APP_TOP%node16.bat
23   call %APP_TOP%node16.bat index.js
24 )
25
26 :LOOP
27
28 rem 監視間隔（秒）の設定（ここでは5秒）
29 powershell -Command "Start-Sleep -Seconds 5"
30
31 rem pid.txtファイルがある限り監視を続ける
32 if exist %CWD%\logs\pid.txt goto EXIST_PID
33
34 rem PIDファイルがない場合、死活監視も任務を終える
35 exit /b 0
36
37 :EXIST_PID
38
39 rem pid.txtの中身を取得(serverのプロセスID)
40 set /p line= < %CWD%\logs\pid.txt
41 echo PID:%line%
42
43 rem tasklistから指定PIDを検索し、存在すれば errorlevelが0になる
44 tasklist /FI "PID eq %line%" | findstr /I "pid" > NUL
45
46 if %errorlevel% equ 0 goto LOOP
47
48 rem サーバーが落ちているとき、起動する
49 goto START
```